IA04: Architecture Decision Record Cart and Checkout Design

ADR-001 Status: Accepted · **Team repo:** https://github.com/AbdiWaris/DE-DCIT208-GROUP-19 · **Video:** {{VIDEO_URL - add after recording}}

Diagram:

Context

The client needs customers to order online without phoning. The shop must never cook an order at the wrong price or twice. Feature: cart (cart.php) and checkout (checkout.php). The decision covers where the cart lives, who is trusted for prices, and how duplicate orders are prevented. The client explicitly sd orders placed by phone or WhatsApp are sometimes forgotten so reliability is the top constrnt.

Assumptions and constrnts

- Customers are mnly on phones with variable mobile data.
- One shop, one server no horizontal scaling needed yet (client mentioned future branches, so this could change).
- Limited budget: no pd external services.
- Team skill is PHP/MySQL; no JavaScript framework experience.
- Delivery addresses and order history are personal data that must be protected.

Alternatives considered

- **A (chosen):** Cart in PHP session; server re-reads prices at checkout from MySQL; one-time checkout_token in session and form blocks duplicates; order saved in one DB transaction.
- **B (rejected):** Cart persisted in a carts database table; unique idempotency key on orders blocks duplicates at DB level.
- **C (rejected):** Cart in the browser (localStorage); JSON sent to an API endpoint.

Decision matrix

Criterion	**Weight**	**A: Session + token**	**B: DB cart + unique key**	**C: Browser cart + API**
Security / no price tampering	4	5 server re-prices (TC-14)	5 - same	2 - browser sends price, must sti re-check
Simplicity / mntnability	3	5 - no extra table or JS client	3 - needs cart cleanup job	2 - needs a second codebase
Team skill match	3	5 - pln PHP sessions	4 - SQL known	2 - JS framework unknown
Deployment cost	2	5 - nothing extra	4	3
Horizontal scale	2	3 -session files = one server	4 - DB cart portable	4
Weighted total		**76**	**64**	**37**

Decision

Option A chosen. Balance: simplicity and security first, accepting weaker horizontal scale. Acceptable because the client has one branch and a limited budget.

Benefits

- No price is trusted from the browser -TC-14 and TC-18 cover this.
- Few moving parts: no cart table, no client-side framework.

On a DB flure the cart and token are preserved customer can retry.

Disadvantages and risks

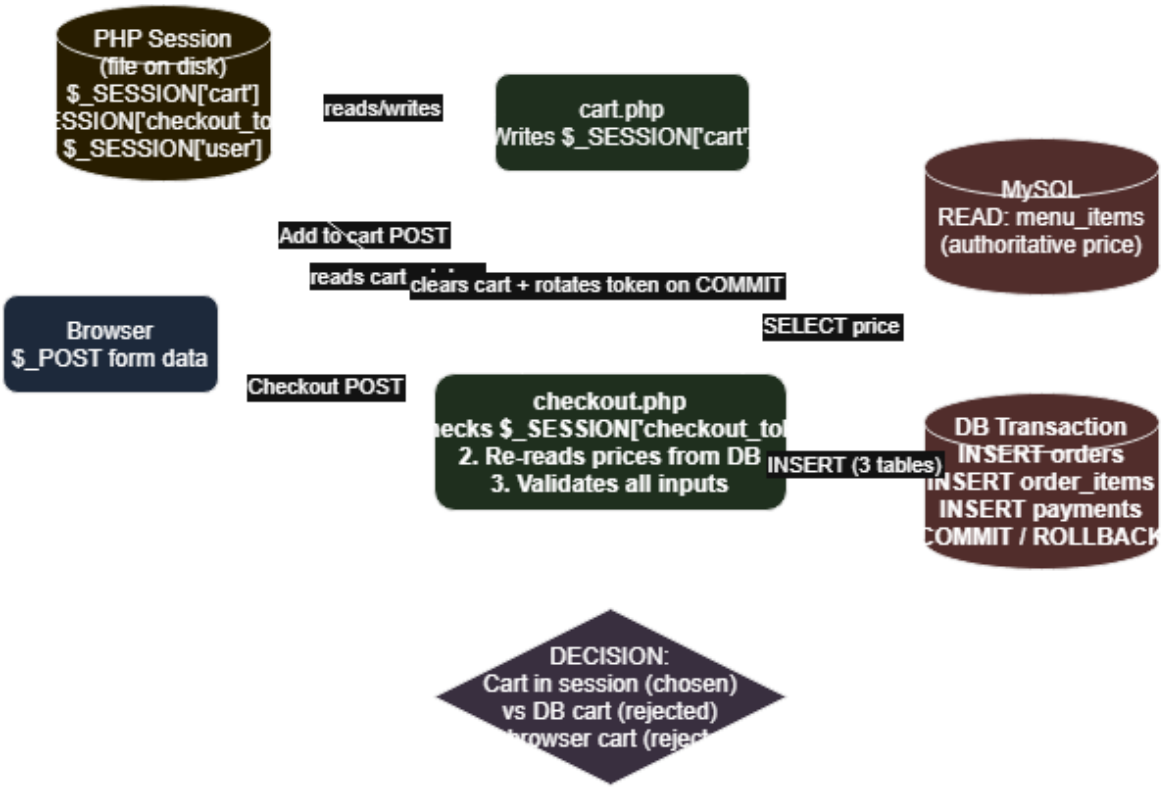
- Cart is lost if the session expires or the customer switches device.

Duplicate protection relies on PHP session locking. If sessions move to a store without per-session locking (e.g. Redis without a lock), two simultaneous submits could both pass the token check - the database has no guard. Fix: add a unique `checkout_token` column on `orders` .

- BUG-07: cart quantity edits are lost because "Continue to checkout" is an <a> link, not a form submit.

Flure walkthrough: database fls during checkout

Step	**What happens**
Normal path	POST → validate → INSERT orders → INSERT order_items → INSERT payments → COMMIT → clear c 302 to my_orders.php
Fling component	MySQL (connection lost, lock timeout, or INSERT fls)
Detection	PHP throws PDOException ; catch (Throwable) in checkout.php runs
User sees	"We could not save the order. Please try agn."
Data state	All three INSERTs rolled back no partial order exists
Logs	error_log('Checkout error: ...') to server log only
Recovery	Cart and token unchanged - customer presses Place order agn


PHP Session
(file on disk)
$_SESSION['cart']
ESSION['checkout_to
$_SESSION['user']
reads/writes
cart.php
Writes $_SESSION['cart']
MySQL
READ: menu_items
(authoritative price)
Add to cart POST
reads cart
clears cart + rotates token on COMMIT
Browser
$_POST form data
SELECT price
Checkout POST
checkout.php
ecks $_SESSION['checkout_to
2. Re-reads prices from DB
3. Validates all inputs
INSERT (3 tables)
DB Transaction
INSERT orders
INSERT order_items
INSERT payments
COMMIT / ROLLBACK
DECISION:
Cart in session (chosen)
vs DB cart (rejected)
browser cart (reject